ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

Cấu trúc Dữ liệu và Giải Thuật - CO2004

Bài tập lớn 3

HỆ THỐNG GỢI Ý ÂM NHẠC CHO BotKify SỬ DỤNG ĐỒ THỊ

Phiên bản 1.0

TP. HỒ CHÍ MINH, THÁNG 04/2026

1 Chuẩn đầu ra

Sau khi hoàn thành bài tập lớn này, sinh viên ôn lại và sử dụng thành thục:

- Thiết kế và hiện thực cấu trúc dữ liệu đồ thị bằng danh sách kề với các thao tác thêm đỉnh, thêm cạnh và truy vấn đỉnh kề.
- Hiện thực thuật toán duyệt theo chiều rộng BFS để duyệt đồ thị và gợi ý bài hát liên quan từ một bài hát khởi đầu.
- Áp dụng bài toán tìm thành phần liên thông để phân cụm các bài hát thành các playlist độc lập.
- Hiện thực thuật toán Dijkstra để tìm đường đi ngắn nhất nhằm chuyển tiếp mượt mà nhất giữa hai bài hát trong đồ thị có trọng số.
- Tính toán bậc vào của đồ thị để nhận diện bài hát tâm điểm mạng lưới.
- Áp dụng thuật toán duyệt theo chiều sâu DFS để phát hiện chu trình trong đồ thị có hướng.
- Tự mô phỏng các cấu trúc dữ liệu phụ trợ như hàng đợi, mảng truy vết và ngăn xếp đệ quy mà không sử dụng thư viện có sẵn.

2 Dẫn nhập

Trong bài tập lớn cuối cùng này, sinh viên sẽ xây dựng BotKify - một hệ thống gợi ý âm nhạc sử dụng cấu trúc dữ liệu đồ thị. Hệ thống mô hình hóa mối quan hệ giữa các bài hát dưới dạng đồ thị có trọng số, trong đó mỗi đỉnh là một bài hát và mỗi cạnh thể hiện mức độ tương đồng giữa hai bài hát. Trên nền tảng đó, sinh viên sẽ hiện thực các thuật toán kinh điển trên đồ thị như duyệt theo chiều rộng BFS, tìm thành phần liên thông và đường đi ngắn nhất Dijkstra để phục vụ các chức năng gợi ý bài hát liên quan, phân cụm playlist tự động và tìm lộ trình chuyển tiếp nhạc mượt mà nhất.

3 Mô tả

3.1 Cấu trúc dữ liệu đồ thị

3.1.1 Edge

Đây là `struct` đại diện cho một cạnh có hướng trong đồ thị. Nó chứa biến `target` lưu trữ định danh của đỉnh đích đến và biến `weight` lưu trọng số của cạnh, thể hiện chi phí hoặc độ lệch khi chuyển tiếp nhạc.

- `T target`: Định danh của đỉnh đích mà cạnh này trỏ tới với kiểu tổng quát `T`.
- `double weight`: Trọng số của cạnh, biểu thị chi phí hoặc mức độ tương đồng giữa hai bài hát.

3.1.2 AdjacencyNode

Khai báo `struct` này đóng vai trò là một nút kề, bao gồm `vertex` là đỉnh hiện tại và một mảng động `vector<Edge> neighbors` chứa tất cả các cạnh xuất phát từ đỉnh đó.

- `T vertex`: Giá trị định danh của đỉnh hiện tại.
- `vector<Edge> neighbors`: Danh sách các cạnh xuất phát từ đỉnh này, mỗi phần tử chứa thông tin đỉnh đích và trọng số.

3.1.3 Lớp Graph

Lớp template tổng quát `Graph<T>` quản lý toàn bộ đồ thị mạng lưới. Lớp này sử dụng `vector<AdjacencyNode> adjList` làm danh sách kề và xây dựng các hàm tìm kiếm tuyến tính như `getVertexIndex` để quản lý việc thêm đỉnh qua `addVertex` và thêm cạnh qua `addEdge`.

1. **Thuộc tính (protected):**
   - `vector<AdjacencyNode> adjList`: Danh sách kề lưu trữ toàn bộ các đỉnh và cạnh kề tương ứng của đồ thị.
2. **Phương thức (protected):**
   - `int getVertexIndex(const T& vertex) const`: Tìm kiếm tuyến tính qua `adjList` để trả về chỉ số của đỉnh `vertex`. Trả về `-1` nếu đỉnh không tồn tại.

3. **Phương thức (public):**

- `void addVertex(const T& vertex)`: Thêm một đỉnh mới vào đồ thị. Kiểm tra bằng `getVertexIndex` để tránh thêm trùng lặp; nếu đỉnh chưa tồn tại, tạo một `AdjacencyNode` mới và đẩy vào `adjList`.
- `void addEdge(const T& source, const T& target, double weight, bool isBidirectional = false)`: Thêm một cạnh có trọng số từ `source` đến `target`. Tự động gọi `addVertex` cho cả hai đỉnh. Nếu `isBidirectional` là `true`, thêm cạnh ngược từ `target` về `source`.
- `vector<Edge> getNeighbors(const T& vertex) const`: Trả về danh sách các cạnh kề của đỉnh `vertex`. Nếu đỉnh không tồn tại, trả về vector rỗng.
- `bool hasVertex(const T& vertex) const`: Kiểm tra đỉnh `vertex` có tồn tại trong đồ thị hay không, trả về `true` hoặc `false`.

3.1.4 Ví dụ minh họa cấu trúc đồ thị

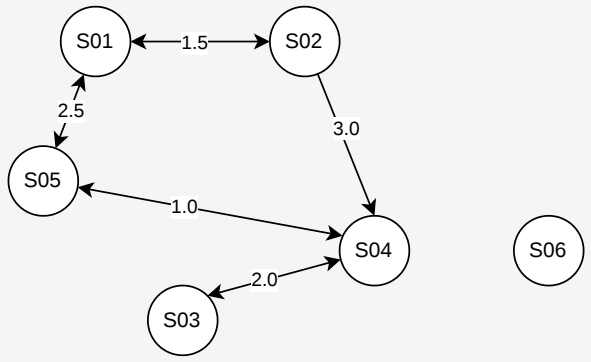
Ví dụ minh họa cấu trúc đồ thị

Giả sử ta xây dựng một đồ thị đơn giản gồm 6 bài hát sau:

ID	Tên bài hát	Nghệ sĩ	Thể loại
S01	Chung Ta Cua Tuong Lai	Son Tung M-TP	Pop
S02	Muon Roi Ma Sao Con	Son Tung M-TP	Pop
S03	Don't Coi	RPT Orijinn	Rap
S04	Thuy Trieu	Quang Hung MasterD	Pop-Dance
S05	Waiting For You	MONO	Pop
S06	Co Don Tren Sofa	Ho Ngoc Ha	Ballad

Các cạnh liên kết được định nghĩa như sau:

- `S01` $\leftrightarrow$ `S02` với trọng số 1.5 loại hai chiều
- `S02` $\rightarrow$ `S04` với trọng số 3.0 loại một chiều
- `S04` $\leftrightarrow$ `S03` với trọng số 2.0 loại hai chiều
- `S01` $\leftrightarrow$ `S05` với trọng số 2.5 loại hai chiều
- `S05` $\leftrightarrow$ `S04` với trọng số 1.0 loại hai chiều
- `S06` không liên kết với bài nào, tạo thành đỉnh cô lập



Hình 1: Ví dụ đồ thị

3.2 Đồ thị Âm nhạc

3.2.1 Giới thiệu

Đồ thị âm nhạc trong BotKify là một đồ thị có hướng và có trọng số. Mỗi bài hát là một đỉnh mang kiểu `string`, và mối liên hệ giữa hai bài hát là một cạnh. Dữ liệu chi tiết của bài hát được định nghĩa thông qua `struct Song` gồm các trường:

- `string id`: Mã định danh duy nhất của bài hát, ví dụ `"S01"`.
- `string title`: Tên bài hát.
- `string artist`: Tên nghệ sĩ trình bày.
- `string genre`: Thể loại nhạc của bài hát.

Ngoài ra, `struct SongEntry` được khai báo `private` bên trong lớp `MusicGraph` đóng gói một cặp gồm:

- `string id`: Mã định danh, dùng làm khóa tra cứu nhanh.
- `Song data`: Đối tượng `Song` chứa toàn bộ thông tin chi tiết của bài hát.

3.2.2 Lớp MusicGraph

Lớp `MusicGraph` kế thừa trực tiếp từ `Graph<string>`. Nó đóng gói mảng `vector<SongEntry> songsList` để lưu trữ dữ liệu thực thể của các bài hát, tách biệt dữ liệu này khỏi cấu trúc mạng lưới đồ thị. Lớp chịu trách nhiệm cung cấp giao diện cho các thao tác nghiệp vụ.

1. **Thuộc tính (private):**

- `vector<SongEntry> songsList`: Mảng lưu trữ danh sách thực thể bài hát, tách biệt khỏi cấu trúc đồ thị kề.

2. **Phương thức (private):**
   - `int getSongIndex(const string& id) const`: Tìm kiếm tuyến tính trong `songsList` để trả về chỉ số của bài hát có mã `id`. Trả về `-1` nếu không tìm thấy.
   - `bool isVisited(const string& id, const vector<string>& visitedList) const`: Kiểm tra xem một đỉnh đã được duyệt hay chưa bằng cách tìm kiếm tuyến tính trong danh sách `visitedList`. Hàm này được tái sử dụng bởi các thuật toán BFS và tìm thành phần liên thông.
3. **Phương thức (public):**
   - `void addSong(const string& id, const string& title, const string& artist, const string& genre)`: Thêm một bài hát mới vào hệ thống. Kiểm tra trùng lặp bằng `getSongIndex`, sau đó tạo đối tượng `Song` và `SongEntry`, đẩy vào `songsList`, đồng thời gọi `addVertex(id)` để thêm đỉnh tương ứng vào đồ thị.
   - `void printSongInfo(const string& id) const`: In thông tin bài hát theo định dạng `[id] title - artist` ra console. Hàm tiện ích được dùng bởi các phương thức khác để hiển thị kết quả.
   - `void recommendRelatedSongs(const string& startId) const`: Gợi ý bài hát liên quan, xem chi tiết tại mục 3.3.1.
   - `void generatePlaylistsByClusters() const`: Tạo playlist theo cụm, xem chi tiết tại mục 3.3.2.
   - `void findSmoothTransition(const string& startId, const string& endId) const`: Chuyển tiếp bài nhạc, xem chi tiết tại mục 3.3.3.
   - `void findMostPopularSong() const`: Tìm bài hát tâm điểm mạng lưới, xem chi tiết tại mục 3.3.4.
   - `void detectMusicLoop() const`: Khám phá vòng lặp âm nhạc, xem chi tiết tại mục 3.3.5.

3.3 Thuật toán hệ thống

3.3.1 Gợi ý bài hát liên quan

Triển khai dựa trên thuật toán tìm kiếm theo chiều rộng BFS. Hệ thống tự mô phỏng hàng đợi bằng một `vector<string> customQueue` kết hợp với con trỏ `head` để duyệt qua các bài hát lân cận và đưa ra gợi ý.

1. **Phương thức:** `void recommendRelatedSongs(const string& startId) const`
2. **Các biến cục bộ chính:**
   - `vector<string> visited`: Danh sách các đỉnh đã được duyệt, giúp tránh lặp vô hạn.
   - `vector<string> customQueue`: Mô phỏng hàng đợi chứa các đỉnh chờ xử lý.
   - `int head`: Con trỏ đầu hàng đợi, tăng dần khi lấy phần tử ra thay vì xóa trực tiếp.
3. **Quy trình hoạt động:**
   - Kiểm tra đỉnh bắt đầu có tồn tại bằng `hasVertex(startId)`.
   - Đưa đỉnh bắt đầu vào `visited` và `customQueue`.
   - Lặp: lấy phần tử tại vị trí `head`, duyệt tất cả `neighbors` qua `getNeighbors`. Với mỗi đỉnh kề chưa thăm, thêm vào `visited` và `customQueue`, đồng thời in thông tin gợi ý bằng `printSongInfo`.

Ví dụ minh họa: BFS

Ví dụ minh họa: Gọi `recommendRelatedSongs("S01")` trên đồ thị mẫu ở mục 3.1.

1. **Khởi tạo:** `visited = [S01]`, `customQueue = [S01]`, `head = 0`.
2. **Bước 1:** Lấy `S01` với head tăng thành 1. Hàng xóm gồm `S02` trọng số 1.5 và `S05` trọng số 2.5. Cả hai chưa thăm nên được thêm vào.
   `visited = [S01, S02, S05]`, `customQueue = [S01, S02, S05]`.
   In: `-> [S02] Muon Roi Ma Sao Con - Son Tung M-TP`
   In: `-> [S05] Waiting For You - MONO`
3. **Bước 2:** Lấy `S02` với head tăng thành 2. Hàng xóm gồm `S01` đã thăm và `S04` chưa thăm nên thêm `S04`.
   In: `-> [S04] Thuy Trieu - Quang Hung MasterD`
4. **Bước 3:** Lấy `S05` với head tăng thành 3. Hàng xóm gồm `S01` và `S04` đều đã thăm nên không thêm gì.
5. **Bước 4:** Lấy `S04` với head tăng thành 4. Hàng xóm gồm `S03` chưa thăm và `S05` đã thăm nên thêm `S03`.
   In: `-> [S03] Don't Coi - RPT Orijinn`
6. **Bước 5:** Lấy `S03` với head tăng thành 5. Hàng xóm `S04` đã thăm nên kết thúc thuật toán.

Kết quả gợi ý theo thứ tự BFS: S02, S05, S04, S03. Bài S06 không được gợi ý vì không liên thông với S01.

3.3.2 Tạo playlist theo cụm

Ứng dụng bài toán tìm thành phần liên thông. Hàm sẽ duyệt qua toàn bộ các đỉnh chưa được thăm lưu trong `visited`, liên tục gọi thuật toán BFS để gom tất cả các bài hát có liên kết với nhau vào chung một cụm playlist.

1. **Phương thức:** `void generatePlaylistsByClusters() const`
2. **Các biến cục bộ chính:**
   - `vector<string> globalVisited`: Danh sách toàn cục theo dõi tất cả các đỉnh đã thuộc về một playlist nào đó.
   - `int playlistCount`: Bộ đếm số playlist được tạo.
   - `vector<string> customQueue` và `int head`: Hàng đợi BFS cục bộ cho từng cụm.
3. **Quy trình hoạt động:**
   - Duyệt lần lượt từng đỉnh trong `adjList`.
   - Nếu đỉnh chưa thuộc `globalVisited`, tạo playlist mới và chạy BFS từ đỉnh đó.
   - BFS gom toàn bộ các đỉnh liên thông vào cùng một cụm, đánh dấu chúng trong `globalVisited` và in thông tin bằng `printSongInfo`.
   - Kết quả sẽ tách các bài hát không liên kết với nhau vào các playlist riêng biệt.

Ví dụ minh họa: Connected Components

Ví dụ minh họa: Gọi `generatePlaylistsByClusters()` trên đồ thị mẫu ở mục 3.1.

1. **Duyệt đỉnh S01:** S01 chưa thuộc `globalVisited` nên tạo **Playlist 1**.
   BFS từ S01 gom được: S01, S02, S05, S04, S03.
   `globalVisited = [S01, S02, S05, S04, S03]`.
2. **Duyệt đỉnh S02, S03, S04, S05:** Tất cả đã thuộc `globalVisited` nên bỏ qua.
3. **Duyệt đỉnh S06:** S06 chưa thuộc `globalVisited` nên tạo **Playlist 2**.
   BFS từ S06 chỉ gom được S06 do không có hàng xóm.

Kết quả in ra:

```
=== Playlist 1 ===
  * [S01] Chung Ta Cua Tuong Lai - Son Tung M-TP
  * [S02] Muon Roi Ma Sao Con - Son Tung M-TP
  * [S05] Waiting For You - MONO
  * [S04] Thuy Trieu - Quang Hung MasterD
  * [S03] Don't Coi - RPT Orijinn
```

```
=== Playlist 2 ===
  * [S06] Co Don Tren Sofa - Ho Ngoc Ha
```

Nhận xét: Bài S06 không có liên kết nào nên tạo thành một playlist độc lập. Các bài S01 đến S05 liên thông với nhau nên thuộc cùng một playlist.

3.3.3 Chuyển tiếp bài nhạc

Sử dụng thuật toán Dijkstra để tìm đường đi ngắn nhất giúp chuyển tiếp mượt nhất giữa hai bài hát. Thuật toán áp dụng phương pháp duyệt mảng truyền thống để tìm đỉnh có khoảng cách nhỏ nhất với độ phức tạp thời gian là $O(V^2)$, sau đó dùng mảng `prev` để truy vết thứ tự phát nhạc.

1. **Phương thức:** `void findSmoothTransition(const string& startId, const string& endId) const`
2. **Các biến cục bộ chính:**
   - `int n`: Số đỉnh trong đồ thị lấy từ `adjList.size()`.
   - `vector<double> dist`: Mảng khoảng cách ngắn nhất từ đỉnh bắt đầu, khởi tạo bằng giá trị `INF = 999999999.0`.
   - `vector<int> prev`: Mảng truy vết, lưu chỉ số đỉnh trước đó trên đường đi ngắn nhất, khởi tạo bằng `-1`.
   - `vector<bool> visited`: Mảng đánh dấu đỉnh đã được xử lý.
   - `vector<string> path`: Mảng lưu đường đi kết quả sau khi truy vết.
3. **Quy trình hoạt động:**
   - Kiểm tra đỉnh bắt đầu và đỉnh kết thúc có tồn tại bằng `getVertexIndex`.
   - Khởi tạo `dist[startIdx] = 0`, các đỉnh còn lại bằng `INF`.
   - Lặp V lần: tìm đỉnh `u` chưa thăm có `dist` nhỏ nhất bằng cách duyệt tuyến tính $O(V)$, đánh dấu `visited[u] = true`.
   - Thực hiện cập nhật khoảng cách: với mỗi cạnh kề của `u`, nếu `dist[u] + weight < dist[v]` thì cập nhật `dist[v]` và gán `prev[v] = u`.
   - Truy vết: từ đỉnh đích, đi ngược mảng `prev` để tái tạo đường đi và in thứ tự phát nhạc bằng `printSongInfo`.

Ví dụ minh họa: Dijkstra

Ví dụ minh họa: Gọi `findSmoothTransition("S01", "S03")` trên đồ thị mẫu ở mục 3.1.

Bảng trạng thái Dijkstra với các đỉnh theo thứ tự adjList: S01, S02, S03, S04, S05, S06:

Bước	dist[S01]	dist[S02]	dist[S03]	dist[S04]	dist[S05]	dist[S06]	Chọn u
Khởi tạo	0	∞	∞	∞	∞	∞	—
1	**0**	1.5	∞	∞	2.5	∞	S01
2	0	**1.5**	∞	4.5	2.5	∞	S02
3	0	1.5	∞	3.5	**2.5**	∞	S05
4	0	1.5	5.5	**3.5**	2.5	∞	S04

Chi tiết quá trình tính toán:

- **Bước 1 với u = S01:** Tính toán `dist[S02] = 0 + 1.5 = 1.5`, `dist[S05] = 0 + 2.5 = 2.5`. Gán `prev[S02] = S01`, `prev[S05] = S01`.
- **Bước 2 với u = S02:** S01 đã thăm. Tính toán `dist[S04] = 1.5 + 3.0 = 4.5`. Gán `prev[S04] = S02`.
- **Bước 3 với u = S05:** S01 đã thăm. Khoảng cách mới tới `S04` là $2.5 + 1.0 = 3.5 < 4.5$ nên cập nhật `dist[S04] = 3.5`, `prev[S04] = S05`.
- **Bước 4 với u = S04:** Đây là đỉnh kề của đích S03. Tính toán `dist[S03] = 3.5 + 2.0 = 5.5`. Gán `prev[S03] = S04`. Thuật toán dừng vì `u == endIdx`.

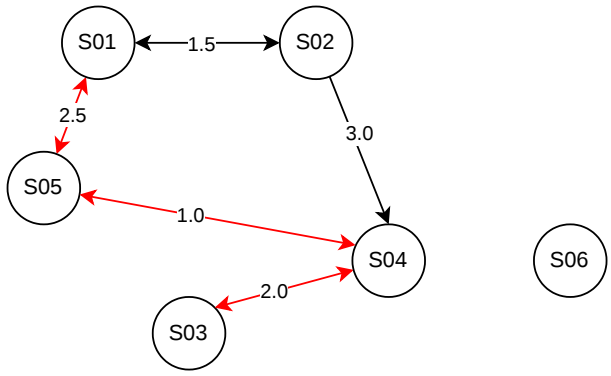
Truy vết ngược mảng `prev` từ S03:

```
S03 <- prev[S03]=S04 <- prev[S04]=S05 <- prev[S05]=S01 (start)
```

Kết quả in ra:

```
-> Tổng độ lệch (Cost): 5.5
-> Thứ tự phát nhạc:
   1. [S01] Chung Ta Cua Tuong Lai - Son Tung M-TP
   2. [S05] Waiting For You - MONO
   3. [S04] Thuy Trieu - Quang Hung MasterD
   4. [S03] Don't Coi - RPT Orijinn
```

Nhận xét: Đường đi ngắn nhất từ S01 đến S03 đi qua S05 rồi tới S04 với tổng chi phí 5.5, thay vì đi qua nhánh S02 có tổng chi phí là 6.5.



Hình 2: Đồ thị âm nhạc sau khi tìm đường đi ngắn nhất

3.3.4 Tìm bài hát tâm điểm mạng lưới

Nhận diện bài hát quốc dân thông qua việc tính toán bậc vào của đồ thị. Hệ thống duyệt qua toàn bộ danh sách kề để đếm số lần một đỉnh xuất hiện ở vị trí đích đến `target`. Đỉnh sở hữu bậc vào cao nhất chính là bài hát mang tính đại chúng, dễ dàng được chọn để phát tiếp từ nhiều bài hát khác trong hệ thống.

1. **Phương thức:** `void findMostPopularSong() const`
2. **Các biến cục bộ chính:**
   - `int n`: Số đỉnh trong đồ thị lấy từ `adjList.size()`.
   - `vector<int> inDegree`: Mảng lưu bậc vào của mỗi đỉnh, khởi tạo bằng `0`.
   - `int maxIdx`: Chỉ số của đỉnh có bậc vào lớn nhất.
3. **Quy trình hoạt động:**
   - Khởi tạo mảng `inDegree` kích thước n với giá trị `0`.
   - Duyệt qua từng đỉnh i trong `adjList`. Với mỗi cạnh kề `Edge` của đỉnh i, tra cứu chỉ số của `edge.target` bằng `getVertexIndex` và tăng `inDegree` tại chỉ số đó lên 1.
   - Tìm đỉnh có `inDegree` lớn nhất bằng duyệt tuyến tính.
   - In thông tin bài hát tâm điểm bằng `printSongInfo` kèm theo giá trị bậc vào.

Ví dụ minh họa: In-degree

Ví dụ minh họa: Gọi `findMostPopularSong()` trên đồ thị mẫu ở mục 3.1.
Nhắc lại các cạnh có hướng trong đồ thị:

- S01 → S02 trọng số 1.5 và ngược lại S02 → S01
- S02 → S04 trọng số 3.0 đây là loại một chiều

- S04 → S03 trọng số 2.0 và ngược lại S03 → S04
- S01 → S05 trọng số 2.5 và ngược lại S05 → S01
- S05 → S04 trọng số 1.0 và ngược lại S04 → S05

Đếm bậc vào dựa trên số cạnh đi đến mỗi đỉnh:

Đỉnh	**Các cạnh đến**	**Bậc vào**
S01	từ S02, từ S05	2
S02	từ S01	1
S03	từ S04	1
S04	từ S02, từ S03, từ S05	**3**
S05	từ S01, từ S04	2
S06	không có	0

Kết quả in ra:

```
-> Bài hát tâm điểm mạng lưới:
   [S04] Thuy Trieu - Quang Hung MasterD
   (In-degree: 3)
```

Nhận xét: Bài S04 Thuy Trieu có bậc vào cao nhất là 3, tức là có 3 bài hát khác có thể chuyển tiếp đến S04. Đây là bài hát quốc dân dễ dàng được phát tiếp từ nhiều nguồn khác nhất trong hệ thống. Bài S06 có bậc vào bằng 0 do là đỉnh cô lập.

3.3.5 Khám phá vòng lặp âm nhạc

Áp dụng thuật toán duyệt theo chiều sâu DFS trên đồ thị có hướng để phát hiện chu trình. Hệ thống sử dụng mảng động để giả lập và theo dõi trạng thái các đỉnh đang nằm trong nhánh đệ quy hiện tại qua mảng `recursionStack`. Nếu quá trình duyệt chạm vào một đỉnh đang nằm trong danh sách theo dõi này, một vòng lặp âm nhạc bất tận sẽ được phát hiện và trích xuất.

1. **Phương thức:** `void detectMusicLoop() const`
2. **Các biến và hàm phụ trợ chính:**
   - `vector<bool> visited`: Mảng đánh dấu đỉnh đã được duyệt qua.
   - `vector<bool> recursionStack`: Mảng theo dõi các đỉnh đang nằm trên nhánh đệ quy hiện tại. Một đỉnh được đánh dấu `true` khi bắt đầu duyệt và `false` khi quay lui.

- `vector<int> parent`: Mảng truy vết, lưu chỉ số đỉnh cha trên đường đi DFS, dùng để trích xuất chu trình khi phát hiện.
- Hàm đệ quy `dfsCycleHelper(int idx, ...)`: Thực hiện DFS từ đỉnh `idx`, đánh dấu `visited[idx] = true` và `recursionStack[idx] = true`. Với mỗi đỉnh kề `v`:
  - Nếu `v` chưa visited: ghi `parent[v] = idx`, gọi đệ quy `dfsCycleHelper`.
  - Nếu `v` đã nằm trong `recursionStack`: phát hiện chu trình, truy vết ngược qua `parent` từ `idx` về `v` để trích xuất danh sách đỉnh tạo thành vòng lặp.

  Khi quay lui, thuật toán đặt lại `recursionStack[idx] = false`.

3. **Quy trình hoạt động:**
   - Khởi tạo `visited`, `recursionStack` kích thước n bằng `false`, mảng `parent` bằng `-1`.
   - Duyệt lần lượt từng đỉnh: nếu đỉnh chưa visited, gọi `dfsCycleHelper` từ đỉnh đó.
   - Khi phát hiện chu trình đầu tiên, trích xuất vòng lặp và in thông tin các bài hát trong chu trình bằng `printSongInfo`, sau đó dừng.
   - Nếu duyệt hết tất cả các đỉnh mà không phát hiện chu trình, in thông báo không tìm thấy vòng lặp.

Ví dụ minh họa: Cycle Detection

Ví dụ minh họa: Gọi `detectMusicLoop()` trên đồ thị mẫu ở mục 3.1.
Khởi tạo: Các mảng `visited` và `recursionStack` đều là false, `parent` điền toàn số -1.

1. **DFS từ S01:** Đánh dấu `visited[S01]=T`, `recStack[S01]=T`.
   Duyệt hàng xóm đầu tiên là `S02`.
2. **DFS từ S02:** Đánh dấu `visited[S02]=T`, `recStack[S02]=T`, `parent[S02]=S01`.
   Duyệt hàng xóm đầu tiên của S02 là `S01`.
   Kiểm tra thấy `S01` đã nằm trong `recursionStack` nên hệ thống **phát hiện chu trình!**
3. **Trích xuất chu trình:** Truy vết ngược từ S02 qua `parent`:
   S02 trỏ về S01, S01 chính là đỉnh gây chu trình nên dừng truy vết.
   Vòng lặp thu được: S01 → S02 → S01.

Kết quả in ra:

```
-> Phát hiện vòng lặp âm nhạc!
-> Vòng lặp:
   [S01] Chung Ta Cua Tuong Lai - Son Tung M-TP
```

```
[S02] Muon Roi Ma Sao Con - Son Tung M-TP
[S01] Chung Ta Cua Tuong Lai - Son Tung M-TP
```

Nhận xét: Chu trình S01 đến S02 rồi quay về S01 tồn tại do cạnh hai chiều giữa hai bài hát này. Thuật toán được thiết kế để dừng ngay khi phát hiện chu trình đầu tiên.

4 Yêu cầu

Để hoàn thành bài tập lớn này, sinh viên phải:

1. Đọc toàn bộ tập tin mô tả này.
2. Tải xuống tập tin initial.zip và giải nén nó. Sau khi giải nén, sinh viên sẽ nhận được các tập tin: main.cpp, main.h, MusicGraph.h, MusicGraph.cpp, và các file dữ liệu đọc mẫu. Sinh viên phải nộp 2 tập tin là MusicGraph.h và MusicGraph.cpp nên không được sửa đổi tập tin main.h khi chạy thử chương trình.
3. Sinh viên sử dụng câu lệnh sau để biên dịch:
   g++ -o main main.cpp MusicGraph.cpp -I . -std=c++11
   Các câu lệnh trên được dùng trong command prompt/terminal để biên dịch và chạy chương trình. Nếu sinh viên dùng IDE để chạy chương trình, sinh viên cần chú ý: thêm đầy đủ các tập tin vào project/workspace của IDE; thay đổi lệnh biên dịch của IDE cho phù hợp. IDE thường cung cấp các nút (button) cho việc biên dịch (Build) và chạy chương trình (Run). Khi nhấn Build IDE sẽ chạy một câu lệnh biên dịch tương ứng, thông thường câu lệnh chỉ biên dịch file main.cpp. Sinh viên cần tìm cách cấu hình trên IDE để thay đổi lệnh biên dịch: thêm file MusicGraph.cpp, thêm option -std=c++11, -I .
4. Chương trình sẽ được chấm trên nền tảng Unix. Nền tảng chấm và trình biên dịch của sinh viên có thể khác với nơi chấm thực tế. Nơi nộp bài trên BKeL được cài đặt để giống với nơi chấm thực tế. Sinh viên phải chạy thử chương trình trên nơi nộp bài và phải sửa tất cả các lỗi xảy ra ở nơi nộp bài BKeL để có đúng kết quả khi chấm thực tế.
5. Sửa đổi các file MusicGraph.h, MusicGraph.cpp để hoàn thành bài tập lớn này và đảm bảo hai yêu cầu sau:
   - Chỉ có một lệnh include trong tập tin `MusicGraph.h` là:
     #include "main.h"
   - Trong tập tin `MusicGraph.cpp`, chỉ có một lệnh include:
     #include "MusicGraph.h"

- Ngoài hai include trên, không cho phép có bất kỳ lệnh include nào khác trong các tập tin này.
- Hiện thực các hàm được mô tả ở các nhiệm vụ trong BTL này.

6. Sinh viên được khuyến khích viết thêm các hàm để hoàn thành BTL này.

5 Nộp bài

Sinh viên chỉ nộp 2 tập tin: MusicGraph.h và MusicGraph.cpp, trước thời hạn được đưa ra trong đường dẫn "Assignment 3 - Submission". Có một số testcase đơn giản được sử dụng để kiểm tra bài làm của sinh viên nhằm đảm bảo rằng kết quả của sinh viên có thể biên dịch và chạy được. Sinh viên có thể nộp bài bao nhiêu lần tùy ý nhưng chỉ có bài nộp cuối cùng được tính điểm. Vì hệ thống không thể chịu tải khi quá nhiều sinh viên nộp bài cùng một lúc, vì vậy sinh viên nên nộp bài càng sớm càng tốt. Sinh viên sẽ tự chịu rủi ro nếu nộp bài sát hạn chót (trong vòng 1 tiếng cho đến hạn chót). Khi quá thời hạn nộp bài, hệ thống sẽ đóng nên sinh viên sẽ không thể nộp nữa. Bài nộp qua các phương thức khác đều không được chấp nhận.

6 Một số quy định khác

1. Sinh viên phải tự mình hoàn thành bài tập lớn này và phải ngăn không cho người khác đánh cắp kết quả của mình. Nếu không, sinh viên sẽ bị xử lý theo quy định của trường vì gian lận.
2. Mọi quyết định của giảng viên phụ trách bài tập lớn là quyết định cuối cùng.
3. Sinh viên không được cung cấp testcase sau khi chấm bài.
4. Nội dung Bài tập lớn sẽ được Harmony với một số câu hỏi trong bài Kiểm tra Cuối kỳ với nội dung tương tự.
5. Nếu sinh viên sử dụng mã nguồn từ các chương trình AI hoặc công cụ tự động tạo mã mà không hiểu rõ về cách hoạt động và ý nghĩa của mã đó, BTL sẽ bị chấm 0 điểm.

7 Harmony cho Bài tập lớn

Bài kiểm tra cuối kì của môn học sẽ có một số câu hỏi Harmony với nội dung của BTL.

Sinh viên phải giải quyết BTL bằng khả năng của chính mình. Nếu sinh viên gian lận trong BTL, sinh viên sẽ không thể trả lời câu hỏi Harmony và nhận điểm 0 cho BTL.

Sinh viên **phải** chú ý làm câu hỏi Harmony trong bài kiểm tra cuối kỳ. Các trường hợp không làm sẽ tính là 0 điểm cho BTL, và bị không đạt cho môn học. **Không chấp nhận giải thích và không có ngoại lệ.**

8 Gian lận

Bài tập lớn phải được sinh viên TỰ LÀM. Sinh viên sẽ bị coi là gian lận nếu:

- Có sự giống nhau bất thường giữa mã nguồn của các bài nộp. Trong trường hợp này, TẤT CẢ các bài nộp đều bị coi là gian lận. Do vậy sinh viên phải bảo vệ mã nguồn bài tập lớn của mình.
- Bài của sinh viên bị sinh viên khác nộp lên.
- Sinh viên không hiểu mã nguồn do chính mình viết, trừ những phần mã được cung cấp sẵn trong chương trình khởi tạo. Sinh viên có thể tham khảo từ bất kỳ nguồn tài liệu nào, tuy nhiên phải đảm bảo rằng mình hiểu rõ ý nghĩa của tất cả những dòng lệnh mà mình viết. Trong trường hợp không hiểu rõ mã nguồn của nơi mình tham khảo, sinh viên được đặc biệt cảnh báo là KHÔNG ĐƯỢC sử dụng mã nguồn này; thay vào đó nên sử dụng những gì đã được học để viết chương trình.
- Nộp nhầm bài của sinh viên khác trên tài khoản cá nhân của mình.
- Sử dụng mã nguồn từ các công cụ có khả năng tạo ra mã nguồn mà không hiểu ý nghĩa.

Trong trường hợp bị kết luận là gian lận, sinh viên sẽ bị điểm 0 cho toàn bộ môn học (không chỉ bài tập lớn).

KHÔNG CHẤP NHẬN BẤT KỲ GIẢI THÍCH NÀO VÀ KHÔNG CÓ BẤT KỲ NGOẠI LỆ NÀO!

Sau mỗi bài tập lớn được nộp, sẽ có một số sinh viên được gọi phỏng vấn ngẫu nhiên để chứng minh rằng bài tập lớn vừa được nộp là do chính mình làm.

——————HẾT——————